Team Flywise: Innovating Travel Recovery

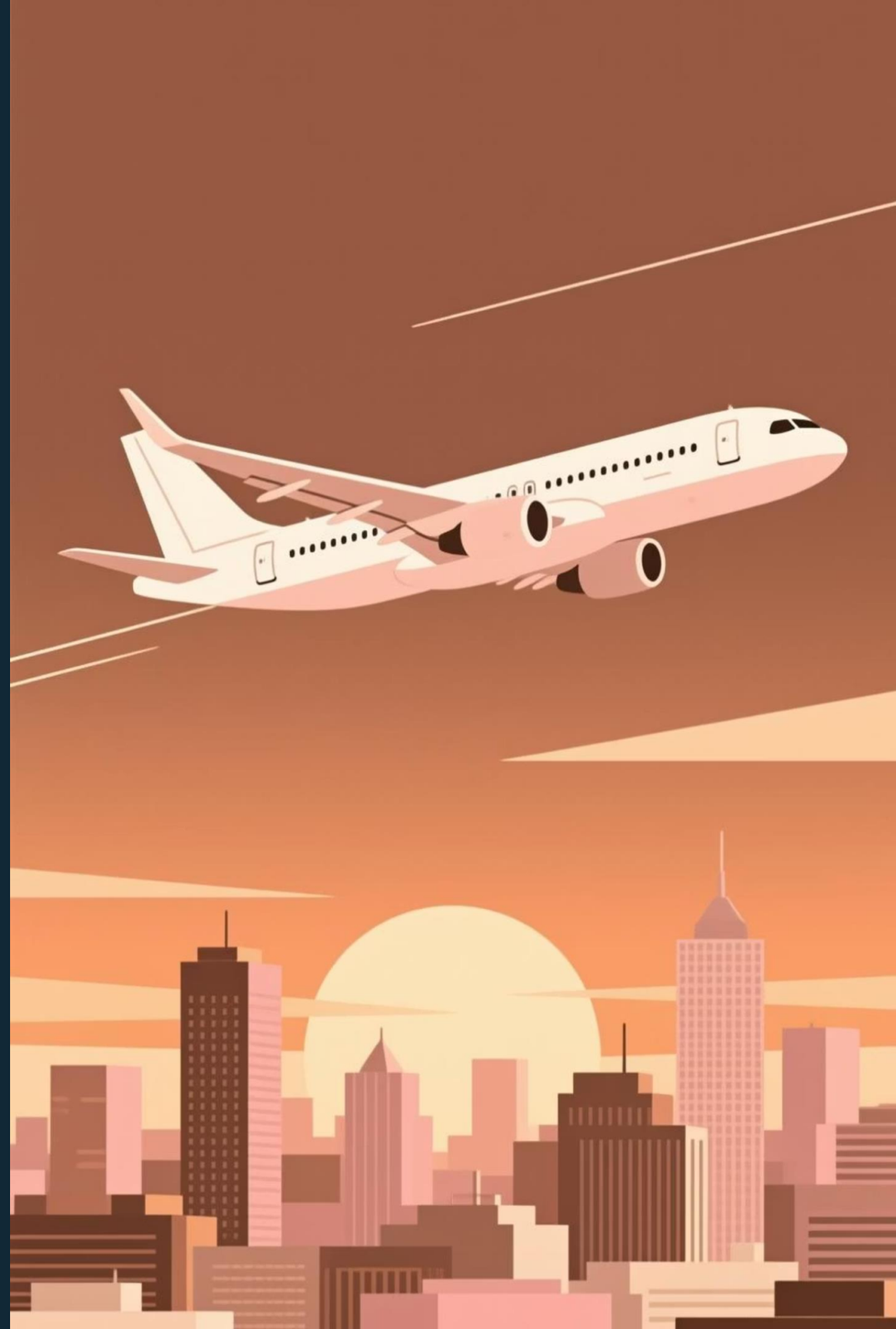
PRESENTED BY

TEAM 9

- Akash Malhotra

- Moinuddin Mohammed

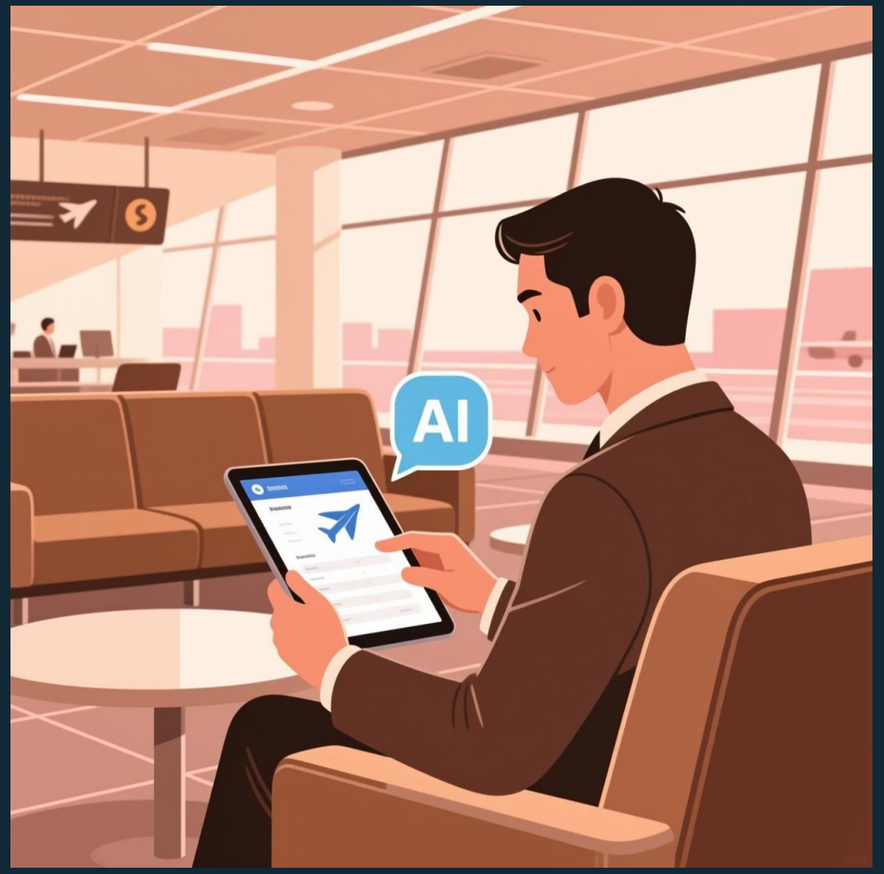
- Nikhil Choudhari



Introduction to Flywise

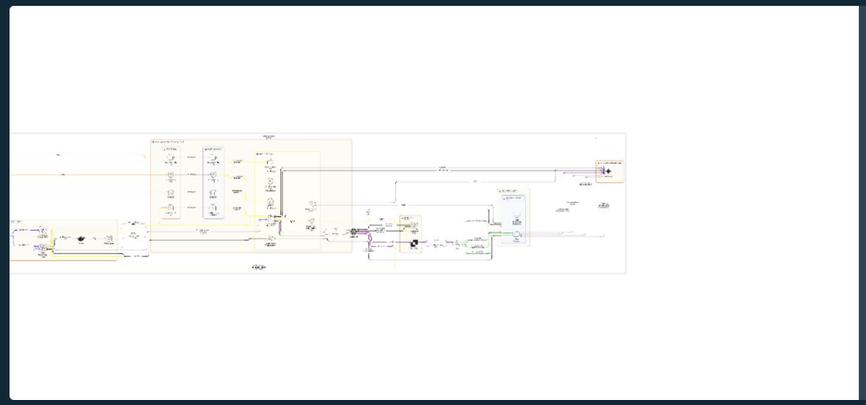
Flywise is an **AI-powered travel disruption assistant** designed to transform stressful flight delays into seamless recovery experiences. It provides real-time monitoring and personalized solutions to keep travelers informed and empowered.

- **Real-time Monitoring:** Actively tracks flight statuses to detect delays as they happen.
- **Personalized Recovery:** Offers tailored options based on user preferences and current situation.



Our key value proposition: Instead of panic during delays, Flywise gives clarity, options, and control, transforming chaos in to a manageable journey.

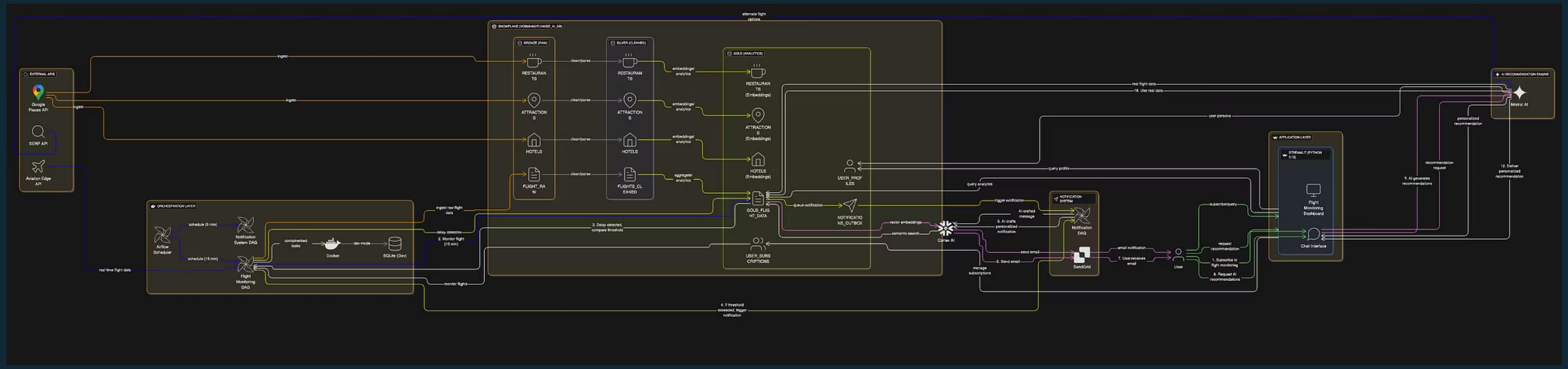
System Architecture



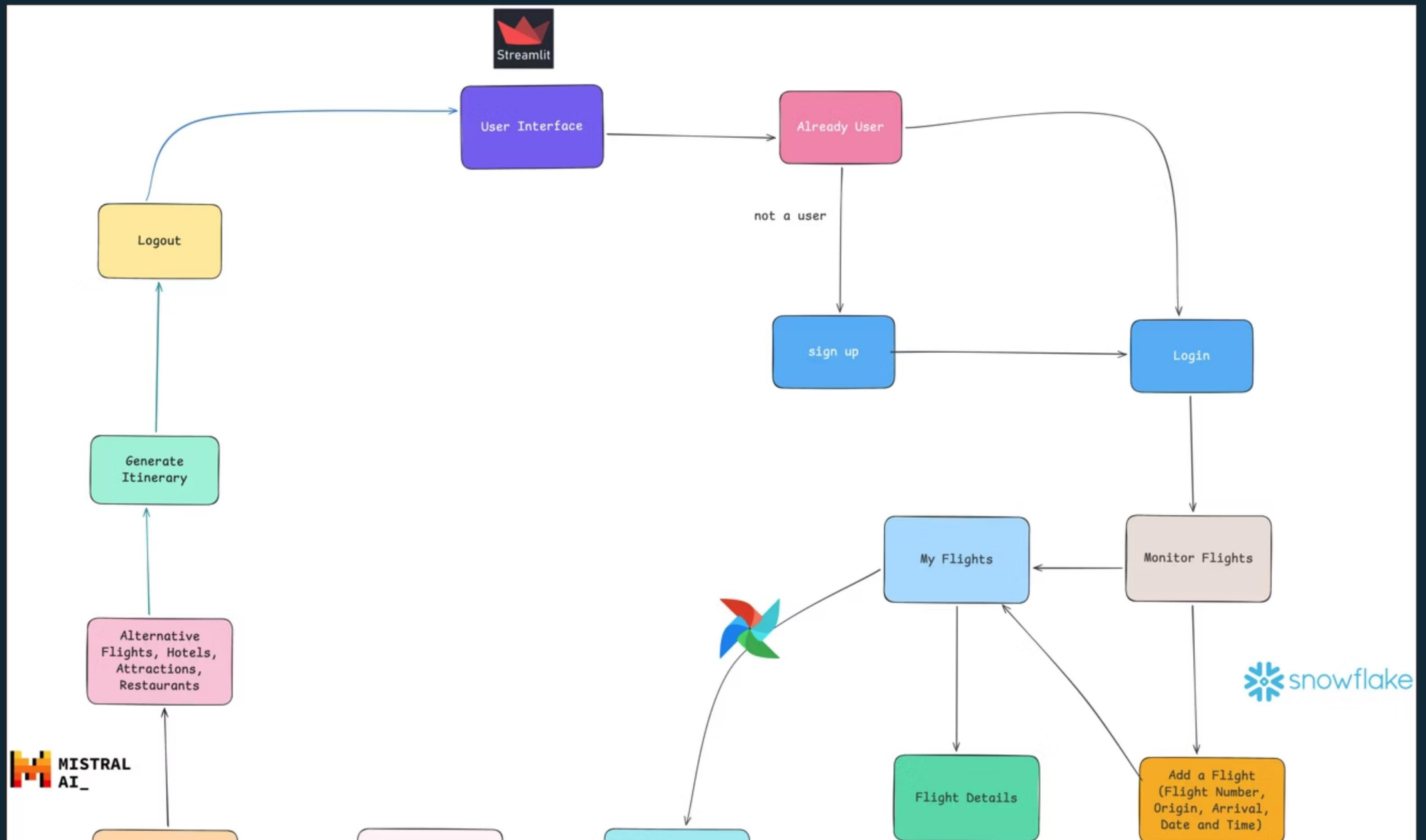
EraserLabs

FlyWise Intelligent Flight Monitoring & Recommendation System Archit

Created with Eraser



User Flow: From Delay to Recovery



Technical Trade-offs: Orchestration & Architecture

Airflow over DBT

Decision: Leveraged Apache Airflow for our data pipeline orchestration, combining ingestion and transformation in a unified workflow.

Why Airflow?

- Enables data ingestion AND transformation in a single pipeline
- Supports Python functions for complex data cleaning logic
- Handles API polling (AviationEdge flight status every 5 minutes)
- Native scheduling with DAGs for time-based triggers

The Trade-off: DBT excels at pure SQL transformations but lacks native data ingestion capabilities. Our pipeline required pulling data from external APIs (AviationEdge, weather services) before transforming - Airflow handles both seamlessly.

Mixtral 8x7B over GPT-4/Claude

Decision: Used native Snowflake Integration with Data Security

Mixtral's Mixture of Experts (MoE) architecture is well-suited for our recommendation tasks:

- Efficient inference (activates only relevant parameters)
- Strong reasoning for travel suggestions
- Good performance on structured data interpretation
- Data never leaves the snowflake boundaries
- Good instruction following
- Fast inference and response times

The Trade-off: Our system is data-intensive with tightly coupled AI pipeline (intent classification → embedding → vector search → LLM), where processing at the data source eliminates network latency and serialization overhead.

Technical Trade-offs: API Data Integration and LLM

Snowflake Arctic Embed M over E5-Base-V2

Decision: We used the Snowflake's Arctic Embed M embedding model

- E5-Base-V2 would require HuggingFace API or AWS SageMaker
- Arctic Embed M model provides a 5x faster user experience
- Our project needs search and retrieval based embeddings for which the Arctic Embed M embeddings are specifically made whereas E5-Base-V2 focuses on general purpose queries
- Our embedding model ranks higher in performance

The Trade off - Although Arctic Embed M is newer with less independent validation compared to E5-Base-V2, it is purpose-built for retrieval tasks

Aviation Edge vs FAA

Decision: We selected AviationEdge as our primary flight status data source instead of FAA's ASWS API.

- Our project needed per flight data
- FAA data covers only the USA
- Aviation Edge Provided comprehensive end-to-end coverage for flight status, delays, and actionable recovery options.

Trade-off: FAA is the official government data source with potentially higher accuracy for US flights. However, the airport-level granularity and US-only coverage make it unsuitable as the primary data source for a flight-specific monitoring system.

Our choices were driven by the need for cost-effectiveness, deep platform integration, and robust data capabilities for real-time travel assistance.

Challenges Faced : Adaptability & Integration

Evolving Project Direction

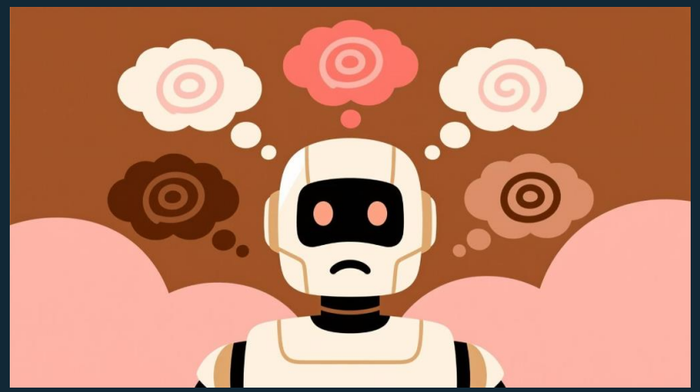
Our project idea shifted twice, necessitating complete redesigns of architecture, data models, and agent logic. This highlighted the critical need for **flexibility in AI-driven projects.**

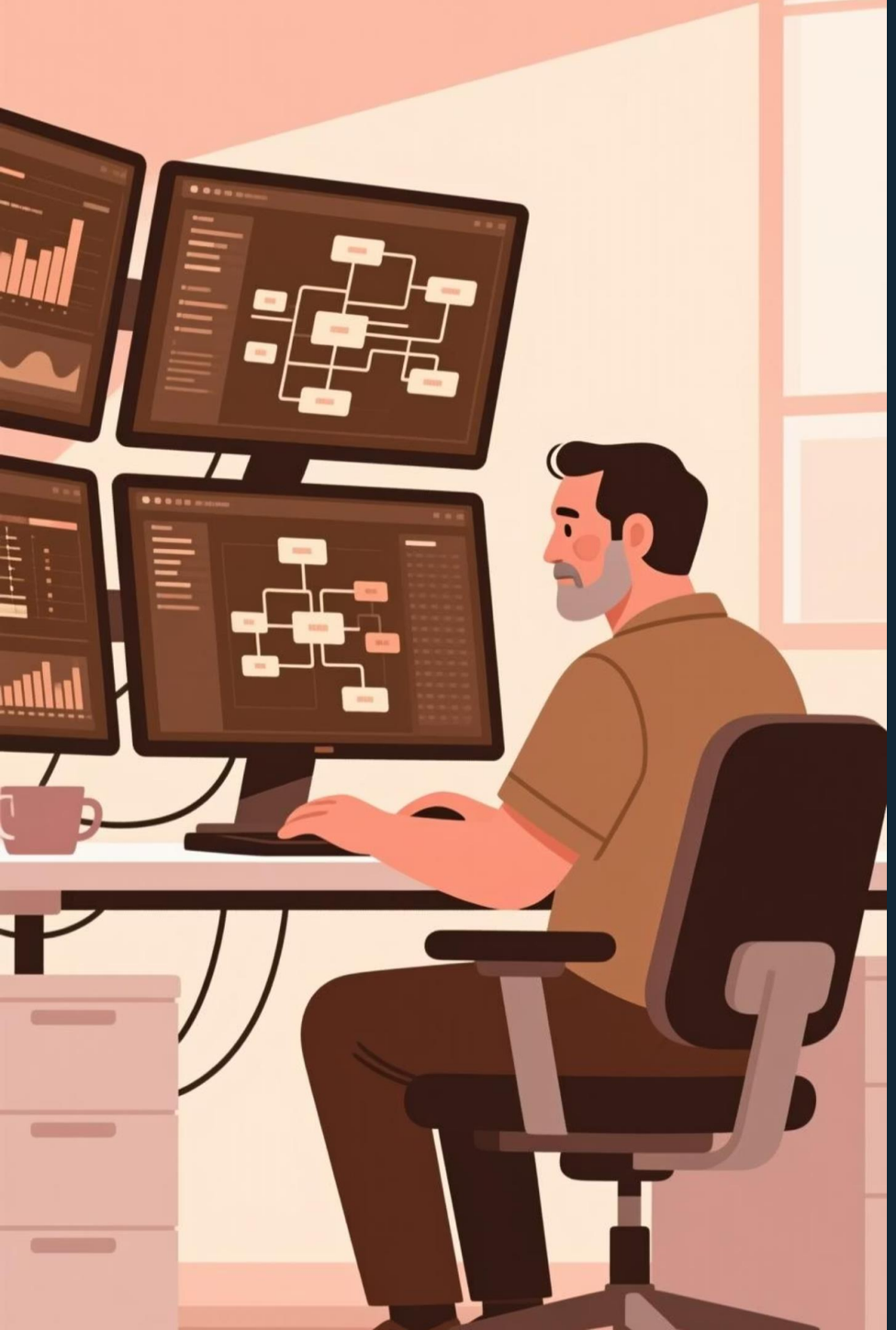
Research Paper Integration

Aligning our implementation with academic research was challenging, as mapping theoretical concepts to real-world constraints often requires significant simplification to be **production-ready.**

Embedding Hallucination

Initial use of embeddings on limited columns led to the agent hallucinating recommendations. The fix involved expanding semantic context and adding **hybrid ranking** (ratings + preferences + embeddings).





Challenges Faced: Technical Implementation

1

Changing Embedding Models

Switching embedding models required re-embedding data and re-tuning similarity logic, underscoring that **embeddings are not plug-and-play** and deeply affect behavior.

2

SERP API Integration in Snowflake

Snowflake's security-first approach meant external API integration was complex, requiring External Access Integrations, Network Rules, and Secrets Management, leading to **time-consuming debugging**.

3

UI Architecture Shift

Initially, we considered external Streamlit microservices but later realized **Snowflake Streamlit could handle everything**, simplifying deployment and reducing moving parts significantly.

Future Scope: The Personal AI Travel Manager

Our vision for Flywise extends beyond delay recovery; we aim to create a comprehensive personal AI travel manager.

ML-based Delay Prediction

Predict delays before they happen, allowing proactive recovery planning.

Booking & Calendar Integration

Direct booking capabilities and auto-import of user flights from calendars.

Mobile App Support

Extend Flywise's capabilities to a dedicated mobile application for on-the-go assistance.

Thank You

Thank you for your time and attention. We are ready to answer any questions you may have about Flywise.